CHOUPI JCVM → Raspberry Pi Pico 2 (Cortex-M33) Port Plan

Oct 9, 2026 · @PetrS

Goal and acceptance criteria

The port is done when an unmodified JCAlgTest v1.8.4 applet CAP is loaded onto a Raspberry Pi Pico 2 running CHOUPI, and the stock JCAlgTest host client, run on a PC, completes the "supported algorithms" test and writes a results file. The plan gets there in phases, and each phase leaves a runnable, testable state behind.

In scope:

- Porting the CHOUPI JCVM (C++) and choupi-os (Rust) to the RP2350 Cortex-M33 cores.
- Building a host-visible card interface, so standard PC/SC tooling sees the Pico 2 as a smart-card reader with a card inserted.
- Adding on-card CAP loading, minimal GlobalPlatform card-manager behaviour, and the Java Card runtime pieces JCAlgTest touches.

Out of scope:

- The Hazard3 RISC-V cores.
- Certification-grade hardening (it has its own deferred track).
- Implementing every cryptographic algorithm.

A working algorithm-support report may legitimately say "not supported" for most algorithms. What matters is that the applet runs, the client completes, and the output is correct for what the platform really offers.

Acceptance criteria:

- [ ] AC1: `gp --list` (GlobalPlatformPro) on the host lists the Pico 2 card's ISD over PC/SC, without custom drivers on Linux and Windows.
- [ ] AC2: The JCAlgTest v1.8.4 release CAP (supported-algorithms applet, Java Card version variant compatible with CHOUPI) installs via `gp --install`, and is selectable by AID.

- [ ] AC3: The JCAlgTest v1.8.4 host client's "supported algorithms" mode runs to completion against the Pico 2 with no transport errors.
- [ ] AC4: The produced results CSV matches what CHOUPI really implements, cross-checked against a reference simulator run (jCardSim) for the same algorithm list.
- [ ] AC5: The result survives a power cycle: the applet stays installed (persistent state in flash) and the test is reproducible.

Starting point

The full CHOUPI source set from April 2020 is recoverable from public mirrors at the exact pinned commits. The upstream `choupi-project` organisation is gone, so the port works from these mirrors.

Component	Language, size	Mirror	Pinned commit
JCVM (`choupi`)	C++17, about 20k lines	gavz/choupi	`fb49537`
OS (`choupi-os`)	Rust nightly-2020, about 8k lines excluding bindings and emulator	gavz/choupi-os	`654f374`
ROM-mask generator (`rommask`)	Java, Maven	gavz/rommask	`ac9f752`
Java Card / GlobalPlatform API (`javacard-api`)	Java, Ant	gavz/javacard-api	`d490faf`

What works today, on the STM32F401 and on the PC emulator:

- All 188 opcodes are decoded and dispatched.
- CAP 2.1 structures are navigated from a pre-linked ROM-mask flash image.
- Stack and heap bounds checks are in place.
- MPU (PMSAv7) compartments isolate applet contexts.
- A log-structured tag-value filesystem runs on internal flash, and the host emulator has unit tests.

What is missing, and blocks the integration test:

- **No APDU path.** There is no ISO 7816 or APDU I/O, so the host cannot talk to an applet. The only I/O is a UART debug console.
- **No applet lifecycle.** There is no applet registry, `install`, `select`, `process`, or `deselect`. The applet ID is hardcoded to 0, and only static entry methods are supported.
- **No on-card loader.** There is no on-card CAP loader or GlobalPlatform card manager. Code exists only in the off-card ROM mask, and only CAP format 2.1 is accepted.
- **No Java exceptions.** `athrow` and handler-table dispatch are unimplemented. JCAlgTest relies on catching `CryptoException` to detect unsupported algorithms, so this blocks it directly.
- **No native API.** All 34 natives are stubs, including `Util.arrayCopy`, the `JCSystem` transient arrays and transactions, and `getAID`.
- **Gaps in persistence and statics.** `putstatic_a` is missing, persistent instance fields are not readable, and the JCVM has no transactions.
- **No crypto API.** The `javacard.security` and `javacardx.crypto` factories are absent, so `getInstance` and `buildKey` cannot even report "not supported".

The toolchain is also stale. It needs Rust nightly-2020-03-10 with `xargo`, and uses 18 old-syntax `asm!` blocks. The Rust OS MPU driver is PMSAv7-only: it relies on power-of-two regions and sub-region disable.

What JCAlgTest v1.8.4 needs from the card

Use `AlgTest_v1.8.4_jc305.cap`. It is CAP format 2.1, which is the only format CHOUPI parses. It also imports API version 1.6, which is exactly what CHOUPI's `javacard-api` build exports.

The v1.8.4 release (29 Sep 2026) ships `AlgTest_dist_1.8.4.zip`. The zip holds `AlgTestJClient.jar` and three CAP variants: jc222, jc304 and jc305. The jc222 variant imports API version 1.3. CHOUPI's import check (`jc_cap_header.hpp`) requires an exact major and minor version match, so the jc222 CAP would fail to link. Relaxing that check to the spec rule (major equal, minor at least the required one) is a small fix, but it isn't needed for the target CAP.

Property	Value in `AlgTest_v1.8.4_jc305.cap`	CHOUPI today
CAP file format	2.1 (compact)	Only 2.1 supported: matches

Property	Value in `AlgTest_v1.8.4_jc305.cap`	CHOUPI today
Package AID	`4A43416C6754657374` ("JCAlgTest")	Needs a package registry and an on-card loader
Applet AID selected by client	`4A43416C675465737431` ("JCAlgTest1"), legacy fallback `6D7970616330303031`	No applet registry or SELECT handling
Imports	`java.lang` 1.0, `javacard.framework` 1.6, `javacard.security` 1.6, `javacardx.crypto` 1.6	Same AIDs and versions exported by the ROM mask: matches
Size	about 31 KB of components (Method component 19 KB)	Fits easily in 4 MB of flash
Transient RAM	600 + 528 bytes `CLEAR_ON_DESELECT`, falling back to `CLEAR_ON_RESET`, then 130 + 8 bytes	Transient arrays not implemented
Extended APDU	Not used (the `ExtendedLength` import is commented out)	Not needed

Host client behaviour. This comes from reading `CardMngr.java` and `SingleModeTest.java`:

1. The client finds readers through `javax.smartcardio` (`TerminalFactory`), so the Pico 2 must appear as a PC/SC reader with a card present.
2. It reads the ATR, then selects the applet by AID.
3. It always sends case-4 APDUs with Le=256, and follows `61xx` with GET RESPONSE. T=1 semantics are therefore simplest.
4. It tries GP GET DATA for CPLC (`9F7F`) and card data (`0066`) after selecting the ISD. Failures are caught and are not fatal, so implementing these is nice-to-have.
5. `-op ALG_SUPPORT_BASIC` and `ALG_SUPPORT_EXTENDED` send `INS_CARD_JCSYSTEM_INFO`, then one `INS_CARD_TESTSUPPORTEDMODES_SINGLE` per algorithm.
6. The client writes `<cardname>_ALGSUPPORT_*.csv` under `-outpath`.

On-card behaviour per test. Each probe calls a factory (`Cipher.getInstance`, `Signature.getInstance`, `KeyBuilder.buildKey`, `KeyPair` generation, `MessageDigest`, `RandomData`, `KeyAgreement`, `Checksum`). It catches `CryptoException` and encodes the

reason code into the response. `process()` also wraps everything in catch blocks that map `CryptoException`, `SystemException`, `NullPointerException` and other exceptions to status words.

The consequence: **Java exception handling, including handler tables and `athrow`, is the single most critical VM feature for this test.** Real crypto is not required. CHOUPI's Java API already throws `NO_SUCH_ALGORITHM` from every factory, which is a correct "not supported" answer.

API surface the applet exercises. Ranked by static call count in the applet source:

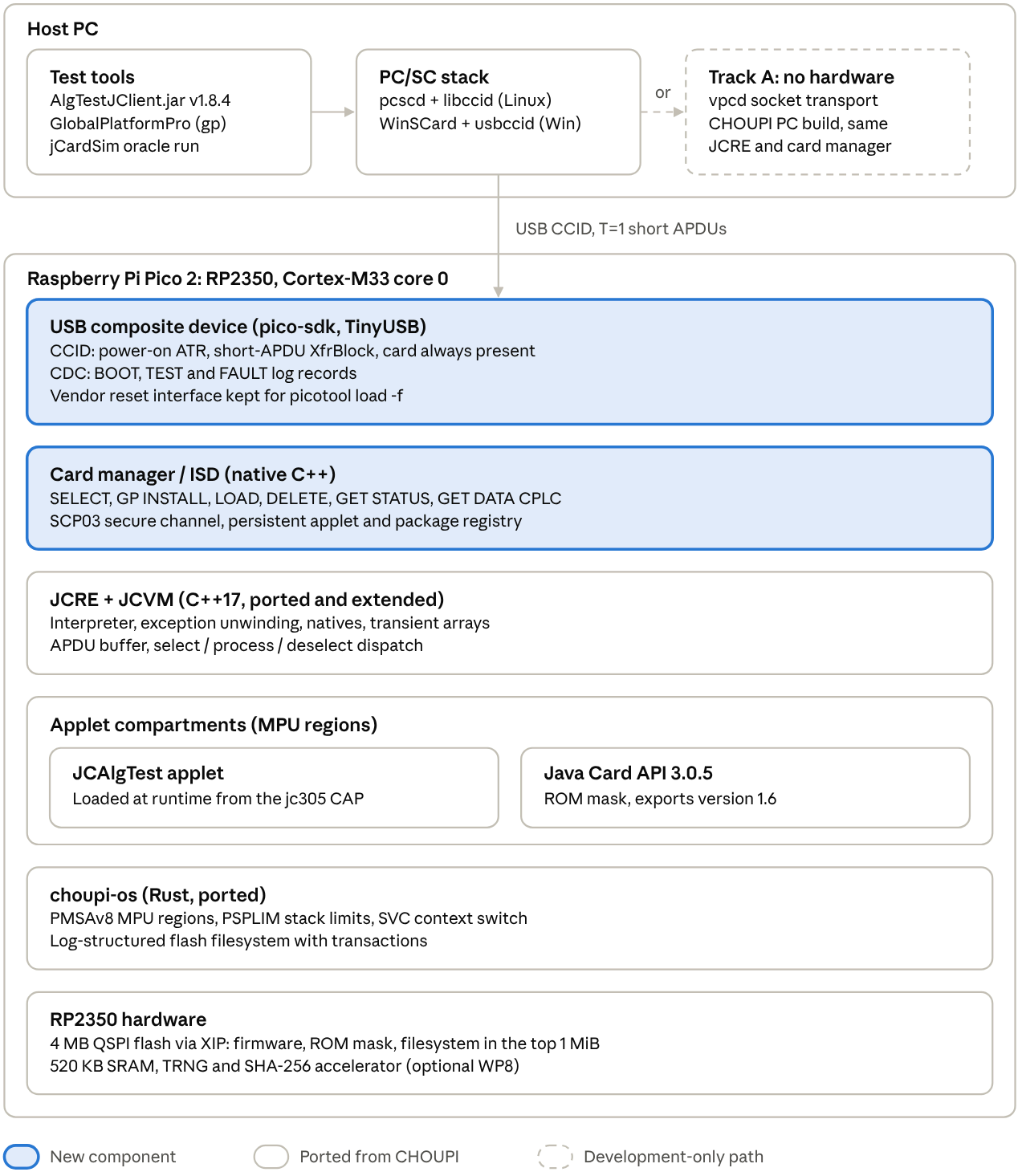
- `Util.setShort/getShort/arrayCopy[NonAtomic]/arrayFillNonAtomic/arrayCompare`
- `ISOException.throwIt`
- `KeyBuilder.buildKey`
- `JCSystem.getAvailableMemory` (short and jc304 array forms), `makeTransientByteArray/ShortArray`, `requestObjectDeletion`, `isObjectDeletionSupported`, `getVersion`, `getMaxCommitCapacity`
- The `getInstance` factories
- `APDU.getBuffer/setIncomingAndReceive/setOutgoingAndSend`
- `Applet.register(byte[],short,byte)` with install parameters

In CHOUPI's Java API, `APDU` is entirely unimplemented: 24 TODOs, and `getBuffer()` returns null. `Applet` has 5 TODOs, `Signature` 15 and `Cipher` 9. The GlobalPlatform card manager (`PlateformeGlobale`) is a skeleton with no APDU dispatch.

Target architecture on the Pico 2

The Pico 2 presents itself to the host as a standard USB CCID smart-card reader with a card always inserted. GlobalPlatformPro and the JCAlgTest client therefore reach CHOUPI through the operating system's own PC/SC stack, with no custom host code. Everything runs on core 0 at the stock 150 MHz. Core 1 stays parked, which keeps flash writes and MPU state simple.

APDUs flow from PC/SC over USB CCID to a native card manager and the JCVM



target architecture · host, USB CCID, card manager, JCVM, OS, hardware

The card manager and the CCID interface are the main new components. The JCVM and choupi-os are ported and extended. The dashed vpcd path lets the same client run against the PC build before the hardware works.

Memory map:

Region	Address range	Contents
Firmware (XIP)	`0x10000000` + about 512 KiB	pico-sdk boot block with `IMAGE_DEF`, the JCVM, and the choupi-os staticlib
ROM mask (XIP)	next 1 MiB	Java Card API packages, starter, static images
Persistent store	top 1 MiB of the 4 MiB	Filesystem: loaded CAPs, static fields, persistent objects, registry, keys
SRAM	`0x20000000`, 520 KiB	OS-private region, shared RW and RO, per-context JCVM stack and heap, transient arrays, USB buffers

The sizes are planning values to confirm in WP1. The filesystem offsets are derived from `PICO_FLASH_SIZE_BYTES`, as `board.md` requires.

How the rp2350-skills are used

The rp2350-skills repo (commit `c989b7e`, 23 Sep 2026) is adopted as the agent and engineering playbook for the whole port. It provides `SKILL.md` plus `reference/board.md`, `reference/silicon.md` and `reference/method.md`, and an `AGENTS.md` workflow. Its rules map onto the work packages as follows.

Skill rule (source)	Applies to	Concrete consequence for this port
Establish board identity first; declare real flash size (`board.md`)	WP1	`PICO_FLASH_SIZE_BYTES=4194304` set before `pico_sdk_init()`. The persistent store sits in the top flash sectors, with offsets derived from the declared size and 4 KiB erase granularity.
`IMAGE_DEF` in first 4 KiB; UF2 family `rp2350-arm-s` (`board.md`)	WP1, WP2	Build on pico-sdk crt0 and boot2. Rust `choupi-os` links as a `staticlib` for `thumbv8m.main-none-eabihf` and is never the image owner.

Skill rule (source)	Applies to	Concrete consequence for this port
Writing own TinyUSB descriptors removes the reset interface (`board.md`)	WP4	The composite USB device keeps the pico-sdk vendor reset interface beside CCID and CDC, so `picotool load -f -x` keeps working without BOOTSEL.
Never write OTP; no `picotool otp` / `erase` in autonomous cycles (`board.md`, `AGENTS.md`)	All; WP9	Secure boot and OTP keys stay out of scope until a manual, human-approved hardening step.
`flash_do_cmd()` stops XIP; flash writes stall both cores (`silicon.md`)	WP3	The `flash_ll` program and erase path runs from SRAM through `flash_safe_execute`. Core 1 stays parked during the JCVM run, and APDU timing tolerates the write stalls.
CDC write blocks up to ~500 ms with no host; CRLF translation corrupts binary (`silicon.md`)	WP1, WP4	No watchdog before boot output. Logs never gate start-up, and CRLF translation is off. CDC carries only text logs, while APDUs travel on CCID.
Machine-verifiable output: `BOOT`, `TEST:PASS/FAIL/SUMMARY`, `FAULT`; source identity in the boot record (`method.md`, `AGENTS.md`)	All	Firmware prints `BOOT id=<git-sha> board=pico2 arch=arm`. Every on-target suite emits `TEST:` records, and a `FAULT` record comes from the MemManage, HardFault and VM-halt paths.
Test without hardware; I/O behind a two-function vtable; pty pairs (`method.md`)	WP4, WP7	The APDU transport is `get_apdu` / `put_response`, with a host build over a pty or socket feeding vsmartcard `vpcd`. The real `AlgTestJClient` therefore runs against the host build before hardware CCID exists.
Bounded build-flash-capture loop; `./tools/build`, `./tools/cycle`, `./tools/monitor`, `./tools/doctor` (`AGENTS.md`)	WP1	Create these wrappers in the port repo, with the effective configuration printed. ARM only is the default; RISC-V builds of platform-independent code are kept warning-free where cheap.

Skill rule (source)	Applies to	Concrete consequence for this port
Small commits, append-only `progress.md`, `.agent/HANDOFF.md` (`AGENTS.md`)	All	Every work-package task closes with a commit and a progress record, so multiple agent sessions can continue the work.

The skills target pico-sdk, Rust `rp235x-hal` and Arduino. Here, pico-sdk is the base for boot, USB, flash and clocks. `rp235x-hal` is used only where the Rust OS needs register definitions for the MPU and SCB, and `cortex-m` crates may suffice for those. The skill set is written for the Feitian RP2350 token as well as the Pico 2. Its "no debug pins on tokens" note matters if a token form factor is a later target: USB CDC is then the only console.

Work packages

The plan runs two tracks in parallel and joins them at the end:

- **Track A** finishes the Java Card runtime on CHOUPI's existing PC build. On the PC, the MPU is emulated by the Rust OS, and APDUs arrive through a vsmartcard bridge.
- **Track B** ports the platform to the Pico 2.

The real JCAlgTest client can run against Track A long before the hardware is ready, so VM bugs and hardware bugs never get debugged together.

WP0: Repository and toolchain baseline (Track A and B)

1. Create the port repo with the four gavz mirrors as submodules at the pinned commits, plus `pico-sdk` (2.x) and `rp2350-skills` as references.
2. Freeze a Docker image with Rust `nightly-2020-03-10`, `xargo`, Boost, Maven and Ant. Build the original PC target unchanged and run the `choupi-os` unit tests. This is the golden baseline.
3. Check that rommask's `capmap-free` dependency (a Bitbucket submodule from the University of Limoges SSD team) is still obtainable. If it is not, vendor a copy, or replace CAP parsing with a small in-house parser.
4. Modernise the Rust build in a separate commit series:
   - Rewrite the 18 old-syntax `asm!` blocks to the current `asm!` syntax.
   - Replace `xargo` with `cargo -Z build-std` (or stable plus `alloc`).

- Drop the vendored `src/std` where possible.
- Re-run the unit tests after each step.

Exit criterion: the PC build runs the original ROM mask, and the OS tests pass on both the old and the modern toolchain.

WP1: Pico 2 bring-up without isolation (Track B)

1. Add a `CHOUPI_TARGET_PICO2` CMake branch using `pico_sdk_init()`, with these settings:
   - `PICO_PLATFORM=rp2350-arm-s` and `PICO_FLASH_SIZE_BYTES=4194304`
   - `PICO_CXX_ENABLE_EXCEPTIONS=1`, because the JCVM signals every error by `throw`
   - `-Os` with LTO kept optional
2. Replace `main_arm.cpp` (STM32 HAL) with `main_pico2.cpp`. It sets up clocks at the stock 150 MHz with `clk_peri` on the 48 MHz USB PLL, a USB CDC log, the LED, and a `BOOT` record carrying the git SHA.
3. Add a single-context shim for the OS FFI: `remote_call` becomes a direct call to `runtime()`, and `fs_*` maps to a RAM-backed or flash-backed store. This proves the JCVM core on M33 before any OS work.
4. Generate the ROM mask for XIP flash (a new `--compactHex` base in the `0x10xxxxxx` range). Add `./tools/build`, `cycle`, `monitor` and `doctor` per the skills.

Exit criterion: the original starter package runs on the Pico 2 and the log shows `BOOT`, then `TEST:PASS jcvm_smoke`.

WP2: choupi-os port to Cortex-M33 (Track B)

Add `src/arch/rp2350/`, mirroring `src/arch/stm32f401re/`. It is about 900 lines today, excluding generated bindings.

1. `mpu_ll`: reimplement for PMSAv8. That means RBAR/RLAR base and limit regions at 32-byte granularity, MAIR attributes, 8 regions, and no overlap or sub-region disable. Replace the power-of-two and alignment assertions in the linker script, and rework the region planner in `src/mpu/` so that compartments are disjoint ranges.
2. `context_ll`, `syscall_ll`, `privilege`: SVC and PendSV stay conceptually the same. Add the ARMv8-M `PSPLIM` and `MSPLIM` stack-limit registers as a free hardware stack-overflow guard.

3. `registers` and `constants`: generate bindings from the CMSIS RP2350 headers in pico-sdk, or use the `cortex-m` and `rp235x-pac` crates.
4. `MemManage_Handler` emits a `FAULT` record before it panics.

Exit criteria:

- The OS unit tests pass on host.
- Positive tests pass on target: two contexts exchange arguments through `argbuf`.
- Negative tests pass on target: an applet-context write to OS-private RAM raises MemManage and logs `FAULT mpu`.

WP3: Flash persistence on QSPI (Track B)

1. Port `flash_ll` to the SDK `flash_range_erase/program`, wrapped in `flash_safe_execute`. The code runs from SRAM and core 1 is never launched.
2. Re-parameterise the filesystem for 4 KiB erase sectors. Place it in the top 1 MiB of the declared 4 MiB, with the ROM mask below it.
3. Implement the filesystem transaction meta-block, which `choupi-os` `TODO.md` already designs. It is the foundation for `JCSystem` transactions and for anti-tearing.
4. Add a power-cut test rig: the host toggles RUN, or uses a USB power switch during writes. Check filesystem consistency after each cut.

Exit criterion: 1,000 randomised power cuts leave the filesystem consistent, and the log shows `TEST:PASS fs_tearing`.

WP4: Host link as a PC/SC reader (Track A and B)

1. Define the APDU transport behind two functions, `get_apdu(buf,&len)` and `put_response(buf,len)`, as the skills recommend.
2. Track A: a host-build transport that speaks the vsmartcard `vpcd` socket protocol. `pcscd` then shows a virtual reader and `AlgTestJClient` runs unmodified against the PC build.
3. Track B: a TinyUSB composite device:
   - A USB CCID interface (class `0x0B`) implemented as a custom class driver: PowerOn returns the ATR; XfrBlock is short-APDU level; GetSlotStatus reports "card always present"; plus the interrupt endpoint.
   - CDC for logs.
   - The pico-sdk vendor reset interface.

4. Choose a T=1 ATR with "JCAlgTest-friendly" historical bytes. Document the Linux and macOS step of adding the VID and PID to libccid's `Info.plist`; existing Pico CCID projects such as pico-openpgp require the same. Windows binds `usbccid.sys` by class.
5. A temporary fallback for early hardware testing: APDU framing over CDC, plus a host bridge to `vpcd`.

Exit criterion: `pcsc_scan` shows the reader and the ATR, and `opensc-tool -s` round-trips an APDU to a native echo handler.

WP5: Java Card runtime completion (Track A, then on target)

1. **Exceptions.** Implement `athrow`, exception-handler-table lookup, and `catch_type` matching through the class hierarchy. Unwinding must work across frames and package-context switches.
   - Map VM-internal C++ exceptions (null pointer, index out of bounds, arithmetic, negative array size, class cast, security) to JCRE-owned Java exception instances, and dispatch them through the same path.
   - An uncaught exception returns SW `6F00` to the host instead of the current `while(1)` halt.
2. **Entry points.** Support virtual entry methods with `this`, needed for `select`, `process` and `deselect`. Support static `install(byte[],short,byte)` with a parameter array.
3. **Natives.** Implement the natives JCAlgTest touches:
   - `makeTransient*Array`, `isTransient`, `getAID`, `selectingApplet`
   - `getAvailableMemory` in both the short and jc304 array forms
   - `isObjectDeletionSupported` (false), with `requestObjectDeletion` as a no-op
   - `getMaxCommitCapacity`, with transaction depth tracking
   - the `arrayCopy`, `arrayFill` and `arrayCompare` natives
   - `clearArray`
4. **Transient memory.** Add `CLEAR_ON_RESET` and `CLEAR_ON_DESELECT` arrays with real clear events. Size the RAM budget so the 600 + 528 byte path succeeds.
5. **Statics and persistence.** Implement `putstatic_a`, persistent instance fields (`JC_Instance::getFields`), and persistent object allocation in the flash store. Objects created in `install()` must survive a reset (AC5). Steps 1 to 4 are enough for AC1 to AC4 on a RAM heap.
6. **Java API (`javacard-api`).**

- Implement `APDU` over natives: a JCRE-owned global APDU buffer, `setIncomingAndReceive`, `setOutgoing*`, `sendBytes*`, and the state machine.
- Implement `Applet.register` (both forms), `AID`, and `JCSystem.getVersion` returning `0x0305`.
- Keep the crypto factories throwing `NO_SUCH_ALGORITHM`, and check that each `getInstance` and `buildKey` path compiles to a clean throw.
- Rebuild the ROM mask.

Exit criterion: on the PC build, a golden test applet exercising try/catch, transient arrays and the APDU echo path passes, followed by the JCAlgTest applet pre-masked into ROM (built to JCA from v1.8.4 source with the converter shipped in the repo).

WP6: Applet lifecycle and card manager (Track A, then on target)

Implement the Issuer Security Domain natively in C++ inside the JCRE. This is faster than finishing the Java `PlateformeGlobale` skeleton, and keeps loading independent of VM correctness.

1. A persistent applet and package registry, with the ISD implicitly selected at power-on.
2. SELECT by AID, with `select()` and `deselect()` callbacks, and `process()` dispatch.
3. GlobalPlatform commands: INSTALL [for load], LOAD (C4 block reassembly), INSTALL [for install and make selectable], DELETE, GET STATUS, and GET DATA for `9F7F` CPLC and `0066`.
4. Secure channel: SCP03 (AES-128, default test keys `40`–`4F`) through mbedTLS. Add SCP02 only if a tool needs it. Security level is authentication only at first.
5. On-card CAP processing, ported from rommask `Filesystem.java`:
   - Validate the components.
   - Resolve imports against registered package exports by AID and version.
   - Strip the StaticField component into per-field filesystem records, initialised from the image.
   - Store the CAP blob under `FILETYPE_CAP`, and update the package list.
   - Run structural checks only; there is no on-card bytecode verifier, so the existing dynamic checks remain the defence.

Exit criterion: `gp --list`, `gp --install AlgTest_v1.8.4_jc305.cap` and `gp --delete` succeed on the PC build through vpcd, then on the Pico 2 through CCID.

WP7: Integration harness (Track A and B)

1. A host script per run:
   - `tools/doctor` checks pcscd, the reader and the BOOT identity.
   - `gp --install` with the release CAP.
   - `java -jar AlgTestJClient.jar -op ALG_SUPPORT_EXTENDED -cardname choupi-pico2 -outpath out/`
   - Archive the CSV, the client log and the device log, with hashes.
2. An oracle run of the same client in `-simulator` mode (jCardSim), plus a diff tool. Every algorithm CHOUPI does not implement must read "not supported" with reason `NO_SUCH_ALGORITHM`. Every implemented one must match the simulator.
3. CI: the PC build plus vpcd on every commit, and the hardware-in-the-loop job on a self-hosted runner with a Pico 2 attached.

Exit criterion: AC1 to AC5 pass three times in a row from a clean flash.

WP8 (optional): Real algorithms

This makes the report non-trivial:

- `RandomData` (`ALG_TRNG`, `ALG_SECURE_RANDOM`) on the RP2350 TRNG
- `MessageDigest.ALG_SHA_256` on the SHA-256 accelerator
- AES, DES, RSA and EC through mbedTLS, behind the same factories, added one algorithm at a time, each with known-answer tests

Test strategy

Every feature is proven first where it is cheapest to debug: host unit tests, then the PC build talking PC/SC through vpcd, and only then the Pico 2. JCAlgTest itself is the final oracle, not the first.

Level	Runs on	What it proves	Pass signal	First needed in
L0 unit	Host (`cargo test`, a C++ test runner)	OS filesystem, flash, MPU and syscall logic; JCVM handlers; CAP parser; exception table lookup	Test-runner exit code 0	WP0
L1 VM golden applets	PC build plus vpcd	try/catch, `athrow` across frames, transient clear events, statics, APDU state machine, `register`	Each applet returns its expected SW and data; host-script diff is empty	WP5
L2 card-manager conformance	PC build, then Pico 2	GP INSTALL, LOAD, DELETE, GET STATUS, SCP03 handshake	`gp --list`, `--install` and `--delete` exit 0; `gp -d` trace archived	WP6
L3 on-target self tests	Pico 2	Boot identity, MPU isolation (positive and negative), flash program and erase from SRAM, filesystem tearing, CCID transport	`BOOT` with expected SHA, then `TEST:SUMMARY pass=N fail=0`; any `FAULT` fails	WP1–WP4
L4 JCAlgTest integration	PC build, then Pico 2	AC1–AC5	CSV produced; diff against jCardSim oracle and expected-support list is empty	WP7

Level	Runs on	What it proves	Pass signal	First needed in
L5 robustness	Pico 2	Power cut during LOAD and during persistent writes; malformed CAP and APDU fuzzing	Card still enumerates; registry consistent; no `FAULT` other than expected SW	WP7

Test assets to build:

- **Golden applets.** About 6 small Java Card applets, compiled to CAP 2.1 against CHOUPI's export files. They isolate one feature each: exceptions, transients, statics, APDU, install parameters, and persistence.
- **A JCAlgTest pre-masked build.** The v1.8.4 applet source converted to JCA and placed in the ROM mask. It runs the client before WP6 exists, which separates "VM cannot run AlgTest" from "loader is broken".
- **A host monitor in the rp2350-skills style.** It requires exactly one `BOOT` with a fresh run ID, then `TEST:` records in order. It fails on timeout, reset or `FAULT`, and archives the log with the firmware hash.
- **Expected-support list.** A text file listing the algorithms CHOUPI claims, initially empty. The L4 diff uses it, so adding an algorithm in WP8 is a one-line, reviewable change.

Security hardening track

Hardening is a separate WP9 that starts after AC1–AC5 pass. Only the items that cost almost nothing if done now, or would be expensive to retrofit, are pulled into the functional work packages.

Pulled into the functional work packages:

Item	Lands in	Why now
Keep all existing `jc_config.h` checks enabled in every build, with a CI check that fails if any security macro is undefined	WP1	They already exist; losing them silently during the port is the main risk

Item	Lands in	Why now
PMSAv8 regions without `PRIVDEFENA` for the kernel (the choupi-os TODO)	WP2	The region planner is being rewritten anyway
`PSPLIM` and `MSPLIM` stack limits per context	WP2	A free hardware guard on ARMv8-M; it costs two registers per context switch
Uncaught exceptions return `6F00` instead of hanging	WP5	Needed functionally, and it removes a denial-of-service hang
PC bounds check on branches and invokes against the current method's code range	WP5	It touches the same interpreter code as exception unwinding
Filesystem transactions and anti-tearing	WP3	A prerequisite for persistent objects and for GP LOAD robustness
Strict CAP structural validation on LOAD; reject unknown or oversized components	WP6	The loader is new code exposed to the host
Default SCP03 keys flagged as test-only; key replacement through PUT KEY	WP6	Avoids a hardcoded-key product

Deferred to WP9, roughly in priority order:

1. Typed operand stack and typed heap fields (`JCVM_TYPED_STACK`, `JCVM_TYPED_HEAP`).
2. A Java Card firewall at object level: owner context checks, plus shareable-interface mediation once shareable interfaces exist.
3. Fault-injection countermeasures:
   - Duplicated or complemented security checks, and hardened boolean and enum encodings.
   - A control-flow step counter.
   - Integrity tags on CAP data and on persistent objects.
   - On the M33, the RP2350 redundancy coprocessor (RCP), for canaries and validated booleans.
4. Platform:
   - Signed boot with OTP keys, SWD disable, and glitch-detector configuration.

    - Encrypted persistent storage.
    - Each of these needs explicit human approval, because OTP writes are irreversible and the skills forbid them in autonomous cycles.
5. Allocator hardening: zeroing on allocate and free, header checksums, and a quarantine list.
6. Fuzzing campaigns on the APDU and CAP parsers, run on the PC build with sanitisers.

The RP2350 is still a general-purpose MCU, not a certified secure element. WP9 can raise the bar substantially, but the result should not be described as a secure element.

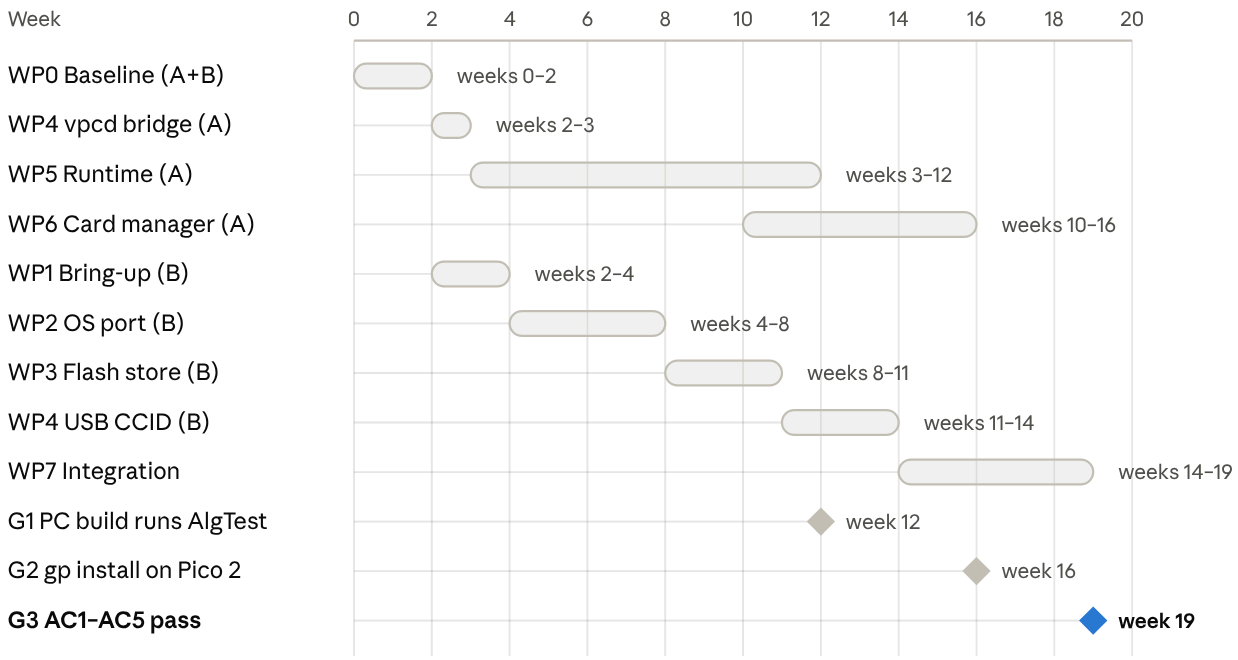
Effort, schedule, risks and decisions

The estimate is 22–33 person-weeks to AC5, excluding the optional WP8 and the WP9 hardening. With two parallel streams, that is about 19 calendar weeks. Track A (runtime and card manager) is the critical path, not the hardware port. The figures are planning estimates, not measurements. They assume one engineer per track, experienced in embedded C++ and Rust, with agent assistance under the rp2350-skills workflow.

The earlier 3–6 week figure for "M33 with MPU isolation" covered only the platform port (WP1–WP2 plus a flash driver). Most of the new work here is the Java Card runtime and card manager that JCAlgTest needs.

AC1–AC5 land in week 19, with the runtime track on the critical path

Estimated weeks from start, two engineers; (A) runtime track, (B) Pico 2 platform track



schedule · 9 work packages, 3 gates, weeks from start

Gate G1 (week 12) is the key de-risking point: the real JCAlgTest client completes against the PC build with the applet pre-masked into ROM. Any VM gap therefore surfaces before the loader and the hardware are involved. G2 (week 16) proves `gp --install` over USB CCID, and G3 (week 19) is AC1–AC5 on the Pico 2.

Risks, highest first:

Risk	Impact	Mitigation
Exception unwinding interacts badly with CHOUPI's package-context list and the C++ exception plumbing	Blocks AC3	Golden applets first (L1); design unwinding on the PC build before touching hardware
rommask transforms CAP data beyond static-field stripping (for example constant-pool rewriting), which the on-card loader must reproduce	Loaded CAP behaves differently from masked CAP	Diff the masked and on-card-processed blobs of the same package byte for byte in L0 tests

Risk	Impact	Mitigation
`capmap-free` (Bitbucket) unavailable	rommask cannot be rebuilt	Check in WP0; vendor a copy or replace it with a small CAP parser
Persistent objects ("object serialisation in flash") are larger work than expected	AC5 slips	AC1–AC4 run on a RAM heap; persistence lands as a separate milestone
CCID host-side driver matching (libccid `Info.plist`) differs per OS	AC1 fails on some hosts	vpcd fallback; document the plist patch; test Linux and Windows early
Flash program stalls and USB timing while LOAD writes flash	Host timeouts during `gp --install`	Buffer LOAD blocks in RAM, commit per block, and keep the CCID time-extension (`RDR_to_PC_DataBlock` with bmCommandStatus time extension)
Rust 2020 code needs more than syntax updates to build on a modern toolchain	WP0 and WP2 slip	Keep the frozen Docker toolchain as a fallback build for the OS staticlib
RAM pressure from C++ STL plus exceptions plus the Rust heap in 520 KB	Transient allocation fails	`getAvailableMemory` telemetry in the `BOOT` record; tune the heap split

Open decisions:

- [ ] D1: Card manager in native C++ (proposed), or finish the Java `PlateformeGlobale`.
- [ ] D2: Secure channel SCP03 only (proposed), or SCP02 as well for older tools.
- [ ] D3: Target JCAlgTest variant: jc305 (proposed), or also relax the version check to accept jc222 and jc304.
- [ ] D4: Keep MPU compartments for the integration milestone (proposed), or run single-context first and add isolation after AC5.
- [ ] D5: USB VID and PID for the CCID device: the Raspberry Pi test PID, or an allocated pid.codes PID.
- [ ] D6: Whether WP8 real crypto is part of the deliverable, or only the "supported algorithms" report on the stub API.

Sources

- JCAlgTest v1.8.4 release. Repository cloned at tag v1.8.4; the CAP manifests, applet source and AlgTest_JClient were inspected locally.
- mcubench/rp2350-skills at commit c989b7e: SKILL.md, AGENTS.md and the three reference files.
- CHOUPI mirrors: gavz/choupi, gavz/choupi-os, gavz/rommask, gavz/javacard-api, inspected at the pinned commits.
- pico-openpgp, referenced for existing RP2350 USB CCID practice, including the libccid Info.plist VID and PID step.
- vsmartcard, for the vpcd virtual reader used in Track A.